College Social Media DB
Assignment Report

Group Name : maaps

March 22, 2026

Contents

1 Assignement Overview **2**

2 Methodology **2**
- 2.1 Module A Methodology . . . 2
- 2.2 Module B Methodology . . . 2

3 Functionality **3**
- 3.1 Module A Functionality . . . 3
- 3.2 Module B Functionality . . . 3

4 Approach **4**
- 4.1 Design Approach . . . 4
- 4.2 Implementation Approach . . . 4

5 Implementation Evidence (Images/Graphs) **5**
- 5.1 Module A Performance Plots . . . 5
- 5.2 Module A B+ Tree Visualizations . . . 6
- 5.3 Module A Benchmark Results Table . . . 6
- 5.4 Module B UI and API Evidence . . . 7
- 5.5 Benchmark and Optimization Evidence . . . 8

6 Tools and External Sources Disclosure **9**
- 6.1 Tools Used . . . 9
- 6.2 External Sources . . . 10

7 Team Member Roles **10**

8 Conclusion **10**

GitHub Repository: Repository

Video Demonstration with Audio Explanation for Module A: VideoLink

Video Demonstration with Audio Explanation for Module B: VideoLink

1 Assignement Overview

This assignment is organized into two modules:

- **Module A:** Data structure and indexing layer with B+ Tree implementation, brute-force baseline, performance benchmarking, and visualization.
- **Module B:** Social media database application layer using FastAPI + MySQL, with authentication, role-based access checks, CRUD operations, and audit/security logging.

2 Methodology

2.1 Module A Methodology

The methodology for Module A was to implement an index-first storage abstraction and evaluate its behavior against a baseline.

- Implement B+ Tree operations: insert, search, delete, range query, update.
- Maintain balancing via split/merge and sibling borrow logic.
- Use a leaf-node linked-list structure for efficient range scans.
- Compare against brute-force implementation using repeatable benchmark settings.
- Measure both execution time and memory peak usage.
- Run tests at various dataset sizes to capture trend behavior.

2.2 Module B Methodology

Module B was implemented as a secure local web application over a relational schema, with API-first enforcement of authentication, authorization, and auditable write operations.

- **Schema-first design:** Separate core identity/auth data from project content data to avoid duplication and preserve normalization.

- **Session-validated APIs:** Protect business endpoints with JWT-based session checks to prevent unauthorized reads/writes.
- **RBAC enforcement:** Apply role and ownership checks at endpoint level (Admin vs Regular User).
- **Auditability:** Log API writes and denied attempts in file logs; track DB write source via trigger-backed logging.
- **Query optimization workflow:** Baseline timings, slow-endpoint identification, targeted index application, and before/after comparison with EXPLAIN.

3 Functionality

3.1 Module A Functionality

- Full B+ Tree implementation with key validation and balancing.
- Range-query support through linked leaves.
- Performance analyzer comparing B+ Tree vs brute-force for:
  - Insert
  - Search
  - Delete
  - Range query
  - Random mixed workload
  - Peak memory usage
- Automated plot generation (JPG) and benchmark JSON export.
- In-memory table and DB manager abstraction for DBMS-style operations.

3.2 Module B Functionality

- **Authentication and session management:** login/signup, JWT session token issuance, token validation for protected routes, and logout flow.
- **Portfolio and social graph:** member portfolio view/update, member search, follower/following retrieval, and follow/unfollow actions.
- **Project-table CRUD:** post creation/listing/view/update/delete and comment creation/listing/update/delete with visibility-aware access.

- **RBAC behavior:** admin-only endpoints for member administration; ownership checks for user modifications.
- **Post permission policy:** only owners can edit posts; Admin can delete any post (including posts authored by other users).
- **Security logging:** API audit log for operations/denials and DB-side write-source tracking for identifying direct unauthorized writes.

4 Approach

4.1 Design Approach

- **Separation of concerns:** module split into indexing/benchmarking and application/security layers.
- **Baseline-driven evaluation:** B+ Tree always compared with brute-force under same workload.
- **Reproducibility:** fixed random seed and scripted benchmark runs.
- **Security-first API design:** credential hashing, token checks, role gates, and logs.

4.2 Implementation Approach

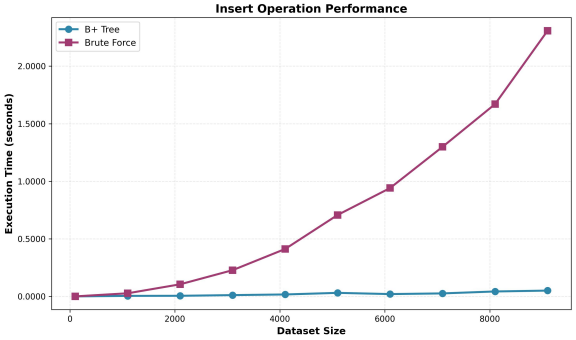
- Module A used Python + Graphviz + Matplotlib for index implementation and result visualization.

For Module B:

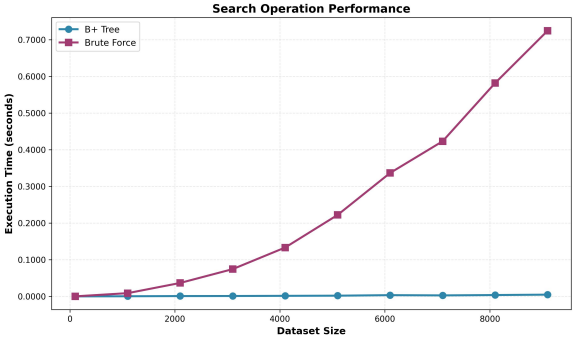
- **Backend stack:** FastAPI + PyMySQL, with SQL schema constraints and trigger-based integrity/logging support.
- **Frontend stack:** static HTML/CSS/JS pages connected to local APIs for end-to-end interaction.
- **Security model:** environment-based JWT secret configuration, bcrypt password hashing, and endpoint-level authorization checks.
- **Performance model:** benchmark logic integrated in notebook workflow for reproducible before/after timing and EXPLAIN evidence.

5 Implementation Evidence (Images/Graphs)

5.1 Module A Performance Plots

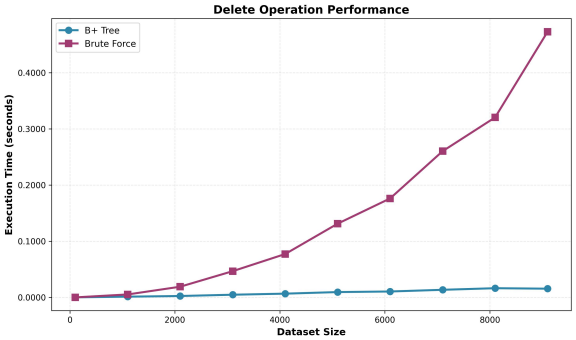


(a) Insert operation

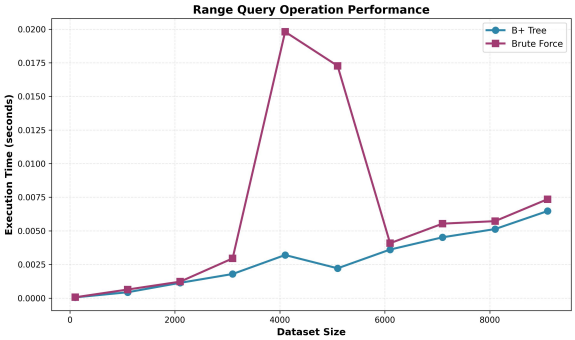


(b) Search operation

Figure 1: Module A: Insert and Search Performance



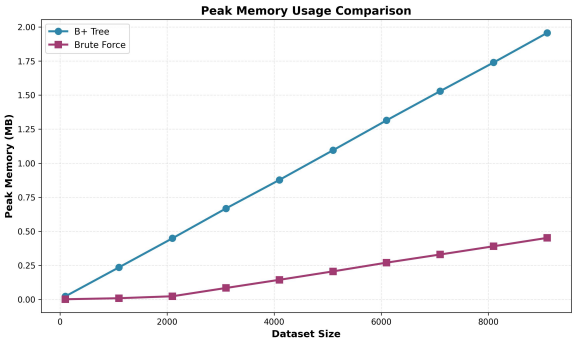
(a) Delete operation



(b) Range query operation

Figure 2: Module A: Delete and Range Query Performance

(a) Random workload



(b) Memory usage

Figure 3: Module A: Workload and Memory Analysis

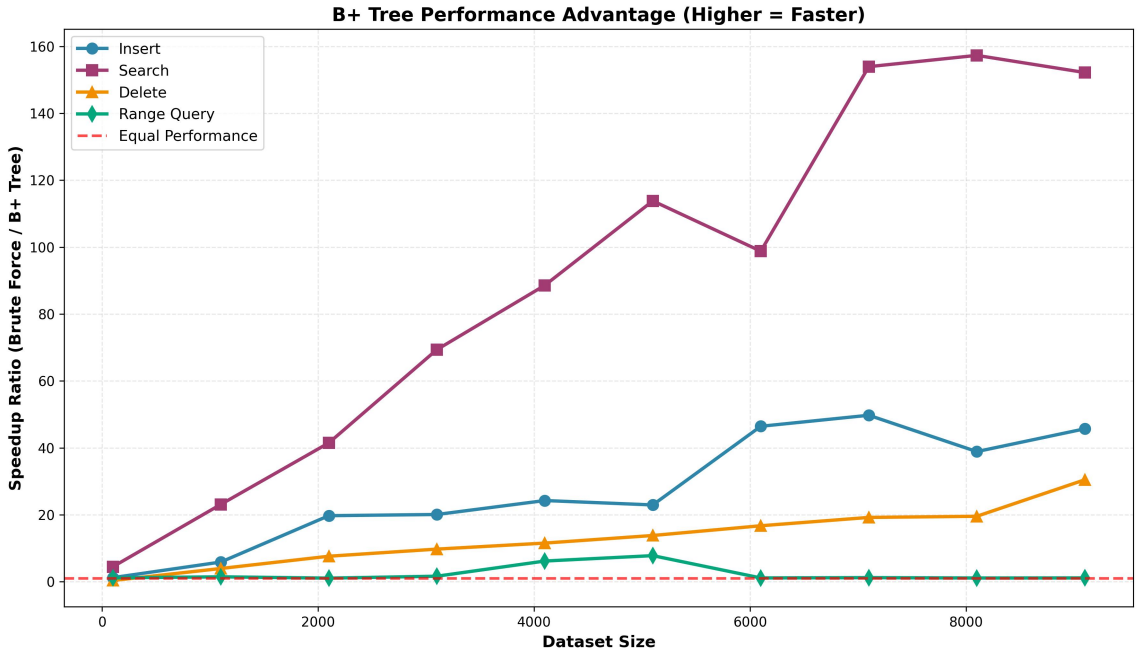
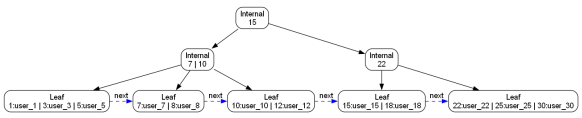
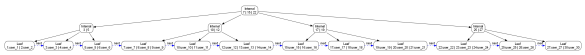


Figure 4: Module A: B+ Tree Speedup Ratio

5.2 Module A B+ Tree Visualizations



(a) B+ Tree demo



(b) B+ Tree large dataset

Figure 5: Module A: B+ Tree Structure Visualizations

5.3 Module A Benchmark Results Table

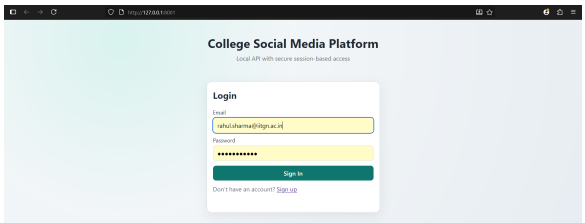
Table 1: Module A benchmark results (rounded to 2 decimals)

Size	Engine	Insert	Search	Delete	Range	Workload	Mem (KiB)
100	B+ Tree	0.00	0.00	0.00	0.00	0.00	20.63
100	Brute Force	0.00	0.00	0.00	0.00	0.00	1.17
1100	B+ Tree	0.01	0.00	0.00	0.00	0.01	255.09
1100	Brute Force	0.04	0.02	0.01	0.00	0.01	9.15
2100	B+ Tree	0.01	0.00	0.01	0.00	0.02	490.58
2100	Brute Force	0.15	0.05	0.08	0.00	0.02	24.15
3100	B+ Tree	0.04	0.00	0.02	0.01	0.04	733.80
3100	Brute Force	0.42	0.12	0.07	0.00	0.07	86.81
4100	B+ Tree	0.03	0.00	0.01	0.00	0.04	967.36
4100	Brute Force	0.65	0.26	0.16	0.00	0.08	149.32
5100	B+ Tree	0.04	0.00	0.02	0.01	0.12	1206.70

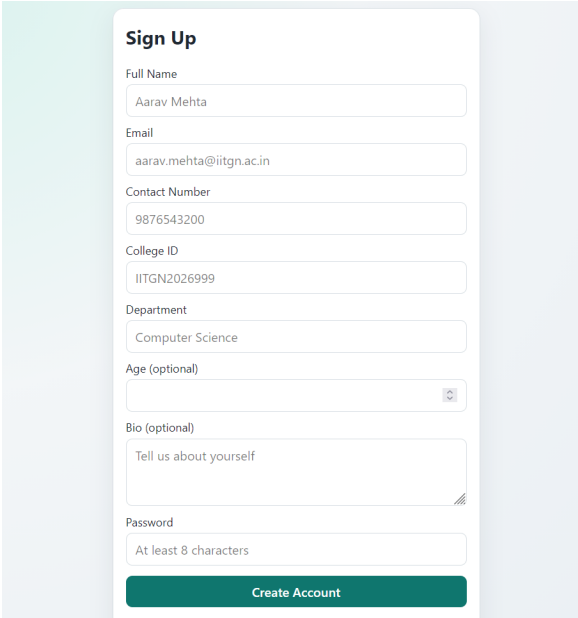
Size	Engine	Insert	Search	Delete	Range	Workload	Mem (KiB)
5100	Brute Force	1.12	0.40	0.21	0.00	0.14	212.12
6100	B+ Tree	0.06	0.01	0.03	0.01	0.09	1448.98
6100	Brute Force	1.63	0.51	0.27	0.01	0.21	422.77
7100	B+ Tree	0.05	0.01	0.03	0.06	0.10	1686.32
7100	Brute Force	1.93	0.70	0.35	0.01	0.27	484.16
8100	B+ Tree	0.06	0.01	0.03	0.01	0.14	1922.76
8100	Brute Force	2.87	1.03	0.63	0.01	0.39	546.45
9100	B+ Tree	0.08	0.01	0.04	0.01	0.16	2165.56
9100	Brute Force	6.21	4.41	2.20	0.01	0.38	609.67

5.4 Module B UI and API Evidence

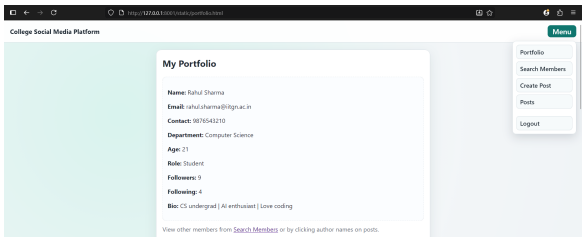
The following screenshots demonstrate login/signup flow, profile management, member search/follow interactions, and post workflows.



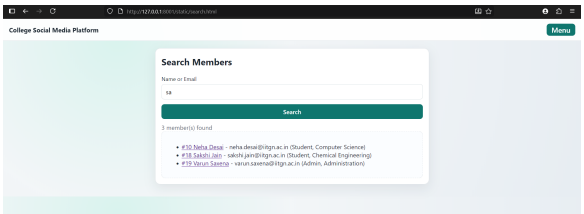
(a) Login page



(b) Signup page

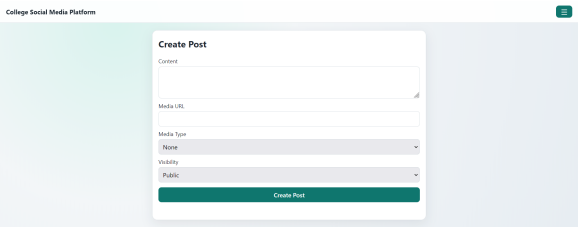


(c) Portfolio/Profile page

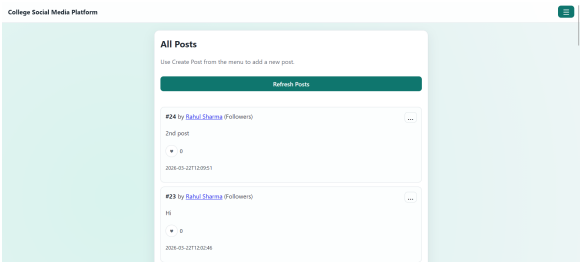


(d) Member search

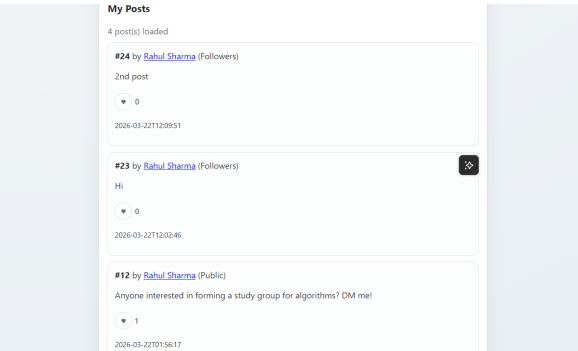
Figure 6: Core authentication and portfolio features (Module B)



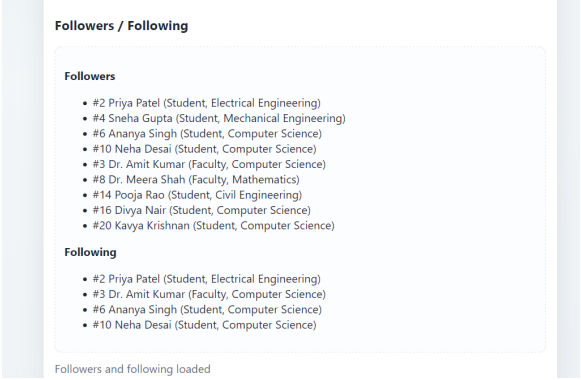
(a) Create post



(b) Posts feed



(c) My posts view



(d) Follow/following interaction

Figure 7: Post and social interaction workflows (Module B)

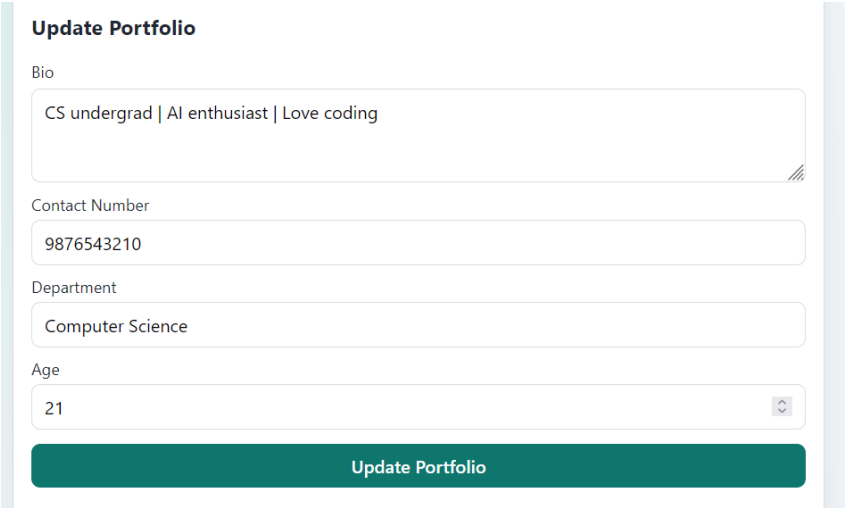


Figure 8: Update options and permission-aware actions

5.5 Benchmark and Optimization Evidence

Quantitative benchmarking and EXPLAIN comparisons are documented in the Module B notebook report and generated benchmark artifact.

- Notebook report: `Module_B/report.ipynb`
- Benchmark artifact: `Module_B/performance/index_benchmark_results.json`

Step 3: Before vs After latency comparison

Endpoint	Layer	Before Avg (ms)	After Avg (ms)	Speedup (x)
GET /posts	SQL	6.164	4.973	1.239
GET /posts/{post_id}/comments	SQL	26.619	20.172	1.320
GET /posts	API	31.097	32.927	0.944
GET /posts/{post_id}/comments	API	89.760	80.823	1.111

Figure 9: Before vs After indexing benchmark comparison for slowest endpoints

Interpretation: The largest gain is on GET /posts/{post_id}/comments at the SQL layer (1.320x). GET /posts shows a smaller gain, which is expected when predicates are less selective. Overall API-level improvements are consistent with the SQL-level reductions.

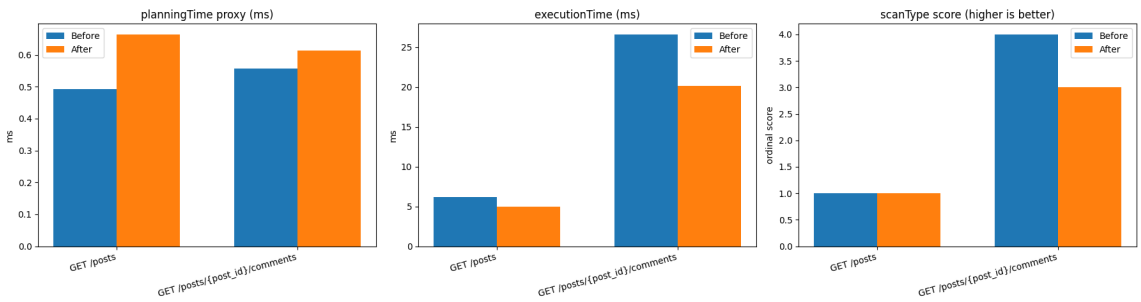


Figure 10: Before vs After planningTime, executionTime, scanType

6 Tools and External Sources Disclosure

6.1 Tools Used

- Python
- FastAPI, Uvicorn
- MySQL, PyMySQL
- Passlib (bcrypt), PyJWT
- Graphviz
- Matplotlib, NumPy
- Jupyter Notebook
- Git/GitHub

6.2 External Sources

- Documentation for Python, FastAPI, MySQL, Graphviz, Matplotlib, and library APIs.
- Used GPT-5.3-Codex for sample data creation and code improvements.

7 Team Member Roles

Team Member	Role / Contribution Area
Saharsh (23110071)	B+ Tree core operations, benchmarking workflow, test execution pipeline, project documentation integration, module A report
N.Eshwar (23110215)	Visualization generation, performance evaluation of bplustree and bruteforceDB, benchmark results, module A report
Manoj (23110025)	Backend API implementation, frontend integration, security/session hardening (Mod B Subtask 1, 2)
Chaitanya (23110052)	RBAC implementation, admin privileges addition in the UI (Mod B Subtask 3) & Report
Praneeth (23110226)	SQL Query Indexing and Optimization, Benchmarking (Mod B Subtask 4, 5)

8 Conclusion

The Assignment delivers:

- an index-optimized data structure module with measurable performance analysis, and
- a secure social-media-style API/database module with role-aware operations and auditability.

The repository includes implementation code, run instructions, generated performance artifacts, and notebook-based evidence for both modules.